Helix OTA — Spec-vs-Implementation Alignment Decision Memo (operational `/api/v1`)

Field	Value
Revision	3
Last modified	2026-06-08T00:00:00Z
Created	2026-06-08
Status	active (decision memo — additive + group breaking WIDENs landed; telemetry/audit breaking WIDENs in progress)
Operator ruling (2026-06-08)	**WIDEN impl to the fuller spec** for the remaining breaking divergences. Executing in sub-batches. **Landed:** group row 6 id-key `id`→`group_id` + row 8 member-add → batch `{device_ids}` → 200 `{added, already_member, not_found}` (device-registration-aware); **row 1 audit `actor` → object `{user_id, subject}`**. Remaining: row 4 (per-device telemetry `events`→`items` + pagination + newest-first), `GET /groups/{id}/members` `items[]` +`added_at` (needs a store membership-timestamp change), row 7 (group list pagination).
Landed (2026-06-08, operator-approved additive WIDENs)	**Row 2 (audit query filters)** — `?since`/`?until` RFC3339 bounds on `GET /audit` (028e656). **Row 5 core (telemetry overview)** — `failure_rate = failure/(success+failure)` + `by_state` (028e656). **Row 6 partial (`member_count`)** — added to `GroupView` on get/list/update (live count via `ListGroupMembers`). All three additive (new optional params / new JSON fields), wire-compatible, real-DB parity where a store change applied. Remaining rows (1 audit actor object, 4 per-device telemetry

Field	Value
	pagination/`events→items`, 8 batch member-add, plus the `id→group_id` rename) are BREAKING wire changes still gated behind an operator WIDEN/TRIM ruling.
Status summary	A per-divergence decision memo over the eight spec-vs-implementation gaps recorded in `implemented_endpoints.md` §10 for the audit-read, telemetry-read, and device-group operational surfaces. For each gap it recommends **WIDEN-impl** (bring the handler up to the fuller `operational_endpoints.md` spec) or **TRIM-spec** (accept the leaner as-built shape as canonical and amend the prose spec), with rationale, effort, risk, and the blast radius (tests / e2e / HelixQA bank). It is a MEMO only — it recommends; it changes no `server/` code, no `go.mod`, no tests, no existing file.
Authority	Recommendations are advisory. The operator decides WIDEN-vs-TRIM per row (Constitution §11.4.66 — closed-set decisions belong to the operator; §11.4.122 — no silent removal of a shipped capability).
Anti-bluff	Per Constitution §11.4.6 (no-guessing): every “impl does” cell was confirmed by direct read of the cited handler function in `server/internal/api/`; every “spec wants” cell was confirmed by direct read of `operational_endpoints.md`. The §10 register at the foot of this memo lists what is **UNVERIFIED** (effort/risk estimates, NFR numbers, store-method availability) and is not asserted as fact.
Owner	Helix OTA control-plane / API team
Related	`implemented_endpoints.md` (as-built §10 gap list — source of this memo); `operational_endpoints.md` (fuller prose spec); `endpoints.md` (conventions/error-model/RBAC/pagination); `openapi.yaml`

table_of_contents

---

1. purpose_and_scope

`implemented_endpoints.md` §10 records eight concrete shape divergences between the prose operational spec (`operational_endpoints.md`) and the code that actually shipped (`handlers_audit.go`, `handlers_telemetry.go`, `handlers_group.go`). The as-built doc states the **code is authoritative** but does not decide which side *should* win going forward. This memo makes that decision per gap:

- **WIDEN-impl** — the spec's fuller shape carries real operational value; raise the handler to it (the as-built doc becomes a transient under-build state, the prose spec stays the target).
- **TRIM-spec** — the leaner as-built shape is sufficient for the 1.0.0-MVP; adopt it as canonical and amend `operational_endpoints.md` + `openapi.yaml` to match (no code churn; doc-sync only).

Scope is exactly the eight §10 gaps across audit / telemetry / groups. Out of scope: rollout (`implemented_endpoints.md` §10 marks it *consistent* with the brick), roles (also *consistent*), and the PLANNED recall endpoint (§8 — a missing feature, not a shape divergence).

2. how_to_read_this_memo

- **`impl does` cites `file:func`** — the exact handler confirmed by reading the source this pass.
- **WIDEN/TRIM** is the recommendation; the operator owns the final call (§11.4.66).
- **Effort** is a relative T-shirt size (S / M / L) for the handler+store+test work; the absolute hours are UNVERIFIED (§9) because the pgx store-method availability for the wider shapes is not confirmed in this memo.
- **Risk** is the chance the change breaks an existing green test or an as-built consumer.
- **Blast radius** names the test layers and HelixQA bank entries that would change. The HelixQA bank path (`tools/helixqa/banks/atmosphere.yaml`) is the constitutional convention (§11.4.58); whether that exact file exists in this repo is UNVERIFIED (§9) — treat it as "the project's Challenge bank, wherever it lives".

3. evidence_basis_confirmed_against_code

Each divergence below was re-confirmed against source this pass (not taken on trust from §10):

- **Audit actor flat-string** — `handlers_telemetry.go` aside, the audit entry is built in `handlers_audit.go:auditMiddleware` (`store.AuditEntry{ActorSubject: claims.Subject, ...}`, L36–45) and rendered by `toAuditLogEntry` into `AuditLogEntry`; the wire `actor` is a single string, never a `{user_id, subject}` object. Spec `operational_endpoints.md` §4.3 shows `"actor": { "user_id": ..., "subject": ... }` (L234).
- **Audit query params** — `handlers_audit.go:handleListAudit` reads only `action`, `resource_type`, `cursor`, `limit` (L133–147). Spec §4.3 (L224–226) lists additionally `?actor`, `?resource_id`, `?since`, `?until`.
- **Audit response type name** — code emits `AuditLogList` (`handleListAudit`, L157); spec §4.3 calls it `AuditList` (L227).
- **Telemetry per-device** — `handlers_telemetry.go:handleDeviceTelemetry` consumes no query params, returns full history in store order under `events` (L51–67); per-event `TelemetryEventView` (L14–22) has no `id`/`success`/`duration_ms`/`bytes_transferred`. Spec §5.1 (L270–292) wants `?limit/?cursor/?event_type/?since/?until/?deployment_id`, newest-first, an `items` array, and those richer fields.
- **Telemetry overview** — `handlers_telemetry.go:handleTelemetryOverview` (L71–82) returns the minimal `TelemetryOverview{EventCounts, Total}` (L32–35), no scoping. Spec §5.2 (L308–342) wants `scope`, `devices_total`, `devices_reporting`, `by_state` (latest-per-device), `failure_rate`, `generated_at`, plus `?deployment_id/?os/?since/?until`.
- **Group view fields + id key** — `handlers_group.go:GroupView` (L28–33) is `{id, name, description, created_at}`; no `filter_criteria`, no `member_count`, id key is `id`. Spec §6.1 (L39–44) shows `group_id`, `filter_criteria`, `member_count`.
- **Group list pagination** — `handlers_group.go:handleListGroups` (L79–90) returns all groups, `GroupList{Items}` (L36–38), no `?name/?limit/?cursor`, no `next_cursor`. Spec §6.2 (L53–54) wants paginated `{items, next_cursor}` + `?name=` prefix.
- **Group member add semantics** — `handlers_group.go:handleAddGroupMember` (L152–171) takes a single `MemberAdd{DeviceID}` (L41–43) and returns `204`. Spec §6.4 (L89–98) wants a batch `{device_ids:[...]}` returning `200` with `GroupMemberAddResult{added, already_member, not_found}`.
- **Group member list shape** — `handlers_group.go:handleListGroupMembers` (L139–150) returns `GroupMembers{group_id, device_ids:[string]}`. Spec §6.4 (L76–85) wants paginated `items` of `{device_id, added_at}`.

4. decision_table

#	divergence	spec wants	impl does (file:func
1	audit `actor` shape	nested `{user_id, subject}`	flat string = subject- (`handlers_audit. toAuditLogEntry`)
2	audit query filters	`?actor`,`?resource_id`,`?since`,`?until` (+ existing)	only `?action`,`?res` (`handlers_audit.`
3	audit response type name	`AuditList`	`AuditLogList` (`handlers_audit.`
4	per-device telemetry: pagination + newest-first + filters + richer fields	`?limit/?cursor/?event_type/?since/?until/?deployment_id`, newest-first, `items[]` with `id/success/duration_ms/bytes_transferred`	no params, full histo lean per-event view (`handlers_teleme TelemetryEventVi

#	divergence	spec wants	impl does (file:func
5	telemetry overview: richer fields + scoping	`scope`,`devices_total`,`devices_reporting`,`by_state` (latest-per-device),`failure_rate`,`generated_at` + `?deployment_id/?os/?since/?until`	minimal {`event_co` (`handlers_teleme` `TelemetryOvervie`
6	`GroupView` fields + id key	`group_id`, `filter_criteria`, `member_count`	`id`, no `filter_crit` (`handlers_group.`
7	`GET /groups` pagination	paginated {`items, next_cursor`} + `?name=` prefix	all groups, no param (`handlers_group.`

#	divergence	spec wants	impl does (file:func
8	group member add: single+`204` vs batch+`200`+result	batch `{device_ids:[...]}` → `200` `{added, already_member, not_found}`	single `{device_id` (`handlers_group. MemberAdd`)

5. per_divergence_rationale

5.1 audit (rows 1–3)

The audit subsystem's only purpose is forensic reconstruction. Two of its three gaps directly defeat that purpose:

- **Row 1 (WIDEN).** The schema keeps a nullable `user_id` precisely so a record outlives the actor (`operational_endpoints.md` §4 intro, `ON DELETE SET NULL`). The handler currently flattens to a single subject string. Surfacing `{user_id, subject}` restores the durable join key for cross-actor queries; effort is small because both values already exist at write time (`claims.Subject` is captured; the `users.id` resolution the spec describes in §4.1 L140 is the only new lookup). **Effort S–M. Risk LOW** — additive JSON field; existing string consumers break only if they assumed `actor` is a string (the as-built doc says it is, so an as-built consumer would break — gate this behind the operator's WIDEN decision).
- **Row 2 (WIDEN).** `?since/?until` time-window scoping is the single most-used audit query in incident response; `?actor`/`?resource_id` complete the "who touched what" triad. The store seam (`store.AuditFilter`) is already the extension point — adding fields there + parsing+validating the RFC-3339 params in `handleListAudit` is the bulk of the work. **Effort M. Risk LOW-MED** — new validation paths (malformed `since/until` → `400`) must be tested to avoid §11.4.1 FAIL-bluffs.
- **Row 3 (TRIM).** Type *name* only; the wire contract (`items`, `next_cursor`) is identical. Renaming `AuditLogList`→`AuditList` is pure churn. Adopt `AuditLogList` as canonical; fix the prose + openapi. **Effort S (doc only). Risk NONE.**

5.2 telemetry (rows 4–5)

- **Row 4 (WIDEN, phased).** Per-device history is unbounded and the handler returns *all of it in insertion order*. At any real device age this is a latency/payload problem and gives no "most-recent-first" affordance. Pagination + newest-first + `?event_type`/`?since`/`?until` filtering are load-bearing and should be widened. The richer fields split: `id` and `success` are cheap derivations; `duration_ms`/`bytes_transferred` require the telemetry ingest to *carry* those values — whether `store.TelemetryRecord` / the `ota-protocol` event already has them is **UNVERIFIED (§9)**. Phase: ship pagination+order+filters first; add the rich numeric fields only once the ingest is confirmed to supply them (else they would be a §11.4-bluff of always-zero fields). **Effort M–L. Risk MED** — touches the store query contract (`TelemetryForDevice` gains filter/cursor params) and changes the response *key* (`events`→`items`), which is a breaking wire change an as-built consumer would feel; gate behind the operator decision.
- **Row 5 (WIDEN core / TRIM `?os`).** `failure_rate` + `by_state` computed as *latest-event-per-device* is the honest fleet-health number; the current raw `event_counts` over-weights chatty devices and has no health ratio at all. Widen to `by_state`/`failure_rate`/`devices_total`/`devices_reporting`/`generated_at`. The latest-per-device aggregate is a `DISTINCT ON (device_id)` query (spec §5.3 alludes to it) — real work in the store. `?os` scoping is the weakest item (needs a device-OS join, low MVP demand) — TRIM it for 1.0.0, keep `?deployment_id` + time window. **Effort L. Risk MED** — additive response fields are safe; the new store aggregate must be tested for the zero-devices edge (`failure_rate` = 0 when `devices_reporting` = 0, per spec §5.2 L345).

5.3 groups (rows 6–8)

- **Row 6 (split).** Three sub-decisions: **id key** `id` (TRIM — matches the rest of the API, renaming to `group_id` churns every group test for cosmetics); **`member_count`** (WIDEN — a cheap COUNT, high-value in list/detail UIs); **`filter_criteria`** (TRIM for MVP — the spec *itself* flags dynamic-membership evaluation as UNVERIFIED and MVP as static-only, §6 intro L13–14; storing+returning an un-evaluated selector is a latent §11.4-bluff "feature that doesn't work", so defer until the evaluator lands). **Effort S–M. Risk LOW.**
- **Row 7 (TRIM, defer).** Groups are operator-curated and bounded (not per-device rows), so an un-paginated list is acceptable at MVP scale. Adopt the lean shape as canonical now; document the paginated `{items, next_cursor}` + `?name=` as a future widen triggered if groups ever get created programmatically. **Effort S (doc). Risk LOW.**
- **Row 8 (WIDEN).** Membership is inherently bulk (build a cohort = add many devices). Single-add forces N round-trips + N audit rows and gives no partial-success feedback; the spec's `{added, already_member, not_found}` is precisely the report an operator needs, and the store method is plural by design (`AddGroupMembers`). The `204`→`200`+body and `device_id`→`device_ids` changes are breaking for an as-built single-add consumer — gate behind the operator decision. **Effort M. Risk MED** — must test idempotent re-add (counted, not error) and unknown-device (`not_found`, partial success) to avoid a §11.4.1 FAIL-bluff on the partial path.

6. overall_recommendation_first_things_first

Order by (operational-harm-if-left-lean) × (low effort/risk first), and honor §11.4.72 only where applicable (no audio surface here, so pure operational priority):

1. **Audit query filters (row 2) — FIRST.** Highest harm: without `?since/?until/?actor/?resource_id` the audit trail cannot answer the incident question it exists for. Medium effort, low-med risk, purely additive query params (no breaking wire change). Biggest forensic value per unit work.
2. **Telemetry overview core fields (row 5, minus `?os`) — SECOND.** `failure_rate` + `by_state` is the fleet-health number operators watch during a rollout; additive response fields are non-breaking. Pairs naturally with the rollout/telemetry story.
3. **Per-device telemetry pagination + newest-first + filters (row 4, fields phase-2) — THIRD.** Prevents the unbounded-history payload cliff; defer `duration_ms`/`bytes_transferred` until the ingest is confirmed to supply them. Breaking key change (`events→items`) means it needs the operator decision + a consumer sweep, so it sits behind the additive wins.
4. **Group member batch-add (row 8) — FOURTH.** Real ergonomics win for cohort building; breaking shape change so it rides with row 4 under the operator decision.
5. **Audit actor object (row 1) — FIFTH.** Restores the durable `user_id` join key; additive but needs the `users` resolution lookup; lower urgency than time-window filtering.
6. **`member_count` on `GroupView` (row 6 partial) — SIXTH.** Cheap UI affordance; do it alongside any group-handler touch.
7. **TRIM-spec items (rows 3, 7, and the `id`-key + `filter_criteria` + `?os` sub-parts of 5/6) — ANYTIME, doc-only.** No code; amend `operational_endpoints.md` + `openapi.yaml` so the prose stops disagreeing with the shipped code (§11.4.6 — divergences must not sit silently). Land these as a single doc-sync change because they cost nothing and immediately stop the spec from lying.

Net: do the three TRIM doc-syncs immediately (zero code, removes the standing divergence), then work the WIDEN list in the 1→6 order above behind a single operator WIDEN/TRIM ruling per breaking row (§11.4.66). Rows 1, 2, 5 are additive/non-breaking and can land without consumer coordination; rows 4, 8 (and the actor-object visibility change in 1) are breaking wire changes that need the as-built consumer sweep first.

7. blast_radius_summary

layer	rows that touch it	what changes
wire-struct unit tests	1,3,4,5,6,8	JSON tag / field-set assertions for `AuditLogEntry`, `TelemetryEventView`, `TelemetryOverview`, `GroupView`, `MemberAdd→batch`, `AuditLogList`/`AuditList` name
	1,2,4,5,8	

layer	rows that touch it	what changes
handler unit/ integration tests		query-param parsing + validation (`400` on bad `since/until/limit`), newest-first ordering, batch idempotency + `not_found` partial success, aggregate edge (`failure_rate` at 0 devices)
store seam tests	1,2,4,5,8	`AuditFilter` (actor/resource/time), `TelemetryForDevice` (filter+cursor), new latest-per-device overview aggregate, `AddGroupMember`→`AddGroupMembers`, `member_count` query
e2e / HelixQA Challenge bank	1,2,4,5,6,8	audit-filter, fleet-health, device-history, group-CRUD, group-membership Challenges each re-assert the new shape with captured evidence (§11.4.69 / §11.4.83)
openapi.yaml	all 8	path/schema definitions re-aligned to whichever side wins each row
audit middleware	8	none — `deriveAuditAction` already emits `group_member` verbs for the members sub-routes (`handlers_audit.go`), so batch-add stays correctly audited

8. doc_sync_actions_if_trim_chosen

For every row the operator rules TRIM-spec, the prose spec and openapi MUST be amended in the same change so they stop disagreeing with the shipped code (Constitution §11.4.6 — no silent divergence; §11.4.65 — keep `.md`/`.html`/`.pdf` siblings in sync):

- Row 3 → `operational_endpoints.md` §4.3: rename response model `AuditList` → `AuditLogList`.
- Row 6 (id) → §6.1/§6.2: change `group_id` → `id` in the `Group`/`GroupView` examples.
- Row 6 (`filter_criteria`) → §6.1/§6.3: mark `filter_criteria` as "stored, MVP returns it but evaluation is deferred — static membership only" (the spec already half-says this at §6 intro).
- Row 7 → §6.2: note `GET /groups` is un-paginated in 1.0.0 (lean shape canonical); keep the paginated form as a documented future-widen.
- Row 5 (`?os`) → §5.2: drop `?os` from the 1.0.0 scope set (keep `?deployment_id` + time window).

This memo does **not** perform those edits — it recommends them pending the operator's per-row ruling.

9. anti_bluff_unverified_register

Per Constitution §11.4.6, the following are explicitly **not asserted as fact** in this memo:

- **UNVERIFIED — effort/risk sizes.** The S/M/L sizes and LOW/MED risk labels are estimates from reading the handlers + store seam, not from a spiked implementation. Absolute hours are not claimed.
- **UNVERIFIED — telemetry rich fields availability (row 4).** Whether `store.TelemetryRecord` / the `ota-protocol` telemetry event already carries `duration_ms` / `bytes_transferred` (and `success` as a first-class field) was **not** confirmed; `handlers_telemetry.go` `TelemetryEventView` does not expose them, and the store record type was not read this pass. If the ingest does not supply them, widening to those fields would produce always-zero columns (a §11.4-bluff) — hence the "phase-2 / defer" qualifier on row 4.
- **UNVERIFIED — store-method availability for wider shapes.** The wider audit filter, paginated telemetry query, latest-per-device overview aggregate (`DISTINCT ON`), `member_count` count, and `AddGroupMembers` plural method are named by the spec (`operational_endpoints.md` §4.2/§5.3/§6.5) but their *existence in the pgx/in-memory `store.Repository`* was not verified here. Effort rises if these must be added to the store.
- **UNVERIFIED — HelixQA bank path.** `tools/helixqa/banks/atmosphere.yaml` is the constitutional convention (§11.4.58); its presence in *this* repo was not confirmed. "Bank" in §4/§7 means the project's Challenge bank wherever it lives.
- **UNVERIFIED — as-built external consumers.** The "breaking wire change" risk on rows 1/4/8 assumes a consumer that already depends on the lean as-built shape. Whether such a consumer (dashboard, device agent) exists and is pinned to the lean shape was not confirmed; the breaking qualifier is a conservative §11.4.6 default (treat as breaking unless proven otherwise), and is exactly why those rows are gated behind an operator §11.4.66 decision and a consumer sweep.
- **FACT (confirmed this pass).** Every "impl does" cell and its `file:func` citation, and every "spec wants" cell, were read directly from the cited source files in `server/internal/api/` and `operational_endpoints.md` during this memo's preparation (see §3).